23

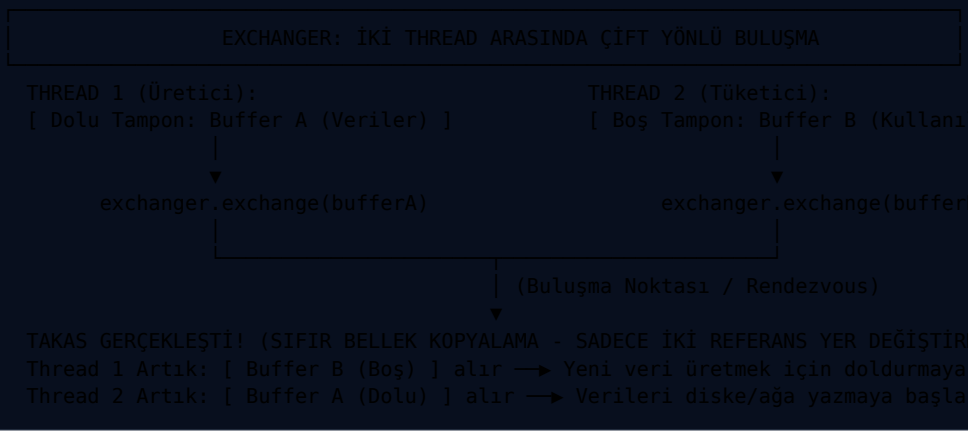
BÖLÜM: İLERI DÜZEY SENKRONIZASYON & KILIT-SIZ ARAÇLAR | MODULE: ADVANCED SYNCHRONIZATION & LOCK-FREE PRIMITIVES

Exchanger: İki Thread Arasında Çift Yönlü Sıfır Kopyalama Veri Takası

Exchanger: Pairwise Zero-Copy Buffer Swapping Between Producer and Consumer

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Casus filmlerindeki köprü sahnesini düşünün. Bir taraftan A ajanı elinde dolu para çantasıyla gelir, diğer taraftan B ajanı elinde boş çantayla gelir. Köprünün ortasında buluşurlar (`exchange()`). Biri gelmeden öbürü çantasını bırakmaz. Tam karşılaştıkları anda çantaları tokuştururlar! A ajanı boş çantayı alıp eve döner, B ajanı para dolu çantayı alıp gider. Bellekte tek bir bayt bile kopyalanmaz; sadece çantalar el değiştirir!

🇬🇧 ENGLISH (ELI5)

Picture a spy bridge exchange. Agent A arrives from the West holding a briefcase packed with cash. Agent B arrives from the East holding an empty briefcase. Neither departs until both arrive at the center span (`exchange()`). At that exact millisecond, they swap cases! Agent A walks away with the empty case to collect more funds; Agent B walks away with the cash. Zero data copying occurs; only two object references instantly swap ownership.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Exchanger Mekanizması ve Ring Buffer Optimizasyonu: 1) Rendezvous (Buluşma) Mantığı: `Exchanger\<V>` iki thread'in çift yönlü senkronize olmasını sağlar. İlk gelen thread, ikinci thread gelene kadar `LockSupport.park()` ile bekletilir. İkincisi geldiğinde referanslar atomik olarak takas edilir ve her iki thread uyandırılır. 2) Sıfır Çöp Üretimi (Zero-Allocation GC Friendly): Yüksek frekanslı finansal sistemlerde (HFT) sürekli `new byte[]` tahsis etmek GC pause yaratır. Exchanger ile önceden ayrılmış iki bellek tamponu (Double Buffering) sürekli el değiştirir; GC baskısı sıfıra iner! 3) Çoklu Çiftler (Arena Desteği): Birden fazla üretici ve tüketici varsa Exchanger içsel olarak bir Arena yapısı kullanarak thread çiftlerini farklı slotlarda eşleştirir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Exchanger Architecture & Cache Locality: 1) Symmetric Rendezvous: Exchanger provides a bidirectional synchronization point. When Thread 1 calls `exchange(V)`, it registers its slot and parks. When Thread 2 arrives, it executes a CAS swapping references, and unparks Thread 1. 2) Eliminating Garbage Collection Churn: High-Frequency Trading (HFT) and telemetry engines avoid constant buffer re-allocations via Double-Buffering. Producer fills Buffer A; Consumer drains Buffer B. At the boundary, they swap references in sub-microsecond time with ZERO heap allocations. 3) Multi-Thread Arena Scaling: Modern JDK Exchangers implement an Arena of slots to eliminate contention when multiple thread pairs exchange concurrently.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Exchanger ile Çift Tamponlu (Double-Buffering) Log Yazıcı
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.Exchanger;

public class DoubleBufferingLogger {
    void main() {
        var exchanger = new Exchanger<List<String>>();

        // Üretici Thread (Log toplayan)
        Thread.ofPlatform().name("LogProducer").start(() -> {
            List<String> currentBuffer = new ArrayList<>();
            try {
                for (int i = 1; i <= 5; i++) {
                    currentBuffer.add("Log Mesajı #" + i);
                    Thread.sleep(50);
                }
                IO.println("Üretici: Tampon doldu, takas bekleniyor...");
                currentBuffer = exchanger.exchange(currentBuffer); // Dolu ver, boş al!
                IO.println("Üretici: Boş tampon alındı, boyut: " + currentBuffer.size());
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        // Tüketici Thread (Diske yazan)
        Thread.ofPlatform().name("LogConsumer").start(() -> {
            List<String> emptyBuffer = new ArrayList<>();
            try {
                IO.println("Tüketici: Veri gelene kadar bekleniyor...");
                List<String> fullBuffer = exchanger.exchange(emptyBuffer); // Boş ver, dolu al!
                IO.println("Tüketici: Dolu tampon alındı! Diske yazılıyor: " + fullBuffer);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Kullanım Kuralı: Exchanger tam olarak İKİ thread arasındaki takaslar için tasarlanmıştır. Eğer 3 thread aynı Exchanger'ı kullanmaya kalkarsa, hangi iki thread'in eşleşeceği öngörülemez ve veri akış sırası bozulabilir.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Design Limitation: Exchanger is explicitly optimized for pairwise (2-thread) cooperation. Utilizing an Exchanger across odd numbers of threads or uncoordinated pools yields non-deterministic pairing, corrupting ordering protocols.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Exchanger, iki thread arasında çift yönlü ve sıfır kopyalama ile veri takas eder.
- Double-Buffering deseninde GC baskısını sıfırlamak için idealdir.
- İlk gelen thread, ikinci thread gelene kadar park edilerek bekletilir.

🎯 Key Takeaways for the Interview

- Exchanger facilitates bidirectional, zero-copy object swapping between two threads.
- Ideal for Double-Buffering architectures to eliminate GC allocation churn.
- The first arriving thread parks until the second arrives, ensuring atomic handoff.